AAROGYA — EMBODIED AI HOSPITAL ROBOTICS PLATFORM

Complete Codebase Architecture & File-by-File Guide

System Scope: Multi-Agent Embodied AI Robot for Autonomous Hospital Assistance using Vision-Language Reasoning
Core Technologies: Python 3.13, FastAPI ASGI, SQLAlchemy 2.0 (SQLite WAL), PyJWT RBAC, NVIDIA Isaac Sim 5.1 (PhysX 5.1), ROS 2 Humble, HTML5 / Vanilla Canvas
Validation Status: 178 Automated Tests Passing (100% Green across 29 Test Suites) | Empirical 2D Log-Odds SLAM Exploration

---

SECTION 1 — 'EVERYTHING IN THE MODEL': THE 6 CORE ARCHITECTURAL PILLARS

In the AAROGYA system, the term **'Model'** refers to six interconnected, mathematically grounded subsystems that cooperate to deliver autonomous embodied assistance in a dynamic healthcare facility. Nothing operates as an isolated black box; every layer has a strictly defined input, state representation, and execution boundary.

- **1. Multi-Agent AI Model** [agents/, multi_agent/]: Coordinates a heterogeneous fleet of 4 Autonomous Mobile Robots (AMRs) using the FIPA Contract Net Protocol. Instead of arbitrary FIFO task queues, robots bid for missions based on battery level, distance, and specialized clinical role (Delivery, Emergency, Reception, Support). Features an automated Priority-1 Code Blue preemption stack where life-critical emergencies instantly suspend and checkpoint routine tasks.
- **2. Vision-Language & NLU Model** [ai_reasoning/, language/, speech/]: Translates unconstrained natural clinical voice audio and text into structured mission payloads. The Web Speech API / Faster-Whisper captures audio; MissionUnderstander regex and semantic parsers extract the clinical intent tuple (Intent, Target Patient, Room ID, Medication, Urgency). Enforces deterministic extraction in < 2ms without cloud LLM hallucinations or HIPAA data leakage.
- **3. Spatial SLAM & Navigation Model** [navigation/, algorithms/, controllers/]: Constructs an empirical 2D log-odds occupancy grid map from 360° planar LiDAR raycasts without synthetic shortcuts. Frontiers (boundaries between free space and unknown territory) are clustered via 8-connected BFS and evaluated via multi-criteria utility scoring: Utility = (Size * 2) / (Dist + 2) - HazardPenalty. The continuous exploration supervisor guides the robot to discover new corridors until completion.
- **4. NVIDIA Isaac Sim 5.1 Digital Twin Model** [isaac_sim_hospital/]: A high-fidelity physics-based digital twin of an 80m x 60m hospital floorplan featuring 18 department suites and 12 moving obstacle agents (doctors, nurses, gurneys, wheelchairs). Steps PhysX 5.1 dynamics at 60 Hz, renders viewports at 30 Hz, and streams 10 Hz non-blocking telemetry over Port 8080.
- **5. Clinical Data & Safety Model** [database/, backend/task_engine.py, backend/auth.py]: A zero-compromise clinical governance engine built on SQLAlchemy 2.0 and SQLite in WAL mode. Enforces PyJWT Role-Based Access Control (RBAC), duplicate delivery lockout (blocking re-orders while a delivery is in transit), bedside RFID/NFC identity verification, and immutable compliance audit trail logging with millisecond timestamps.
- **6. ROS 2 Communication Model** [hospital_robot_nav_ws/, multi_agent/ros2_bridge.py]: The robotics middleware backbone implementing standard DDS topics: /scan (sensor_msgs/LaserScan), /odom (nav_msgs/Odometry), /tf (coordinate transforms: map -> odom -> base_link -> laser_frame), and /cmd_vel (geometry_msgs/Twist) for physical motor actuation.

KEY ARCHITECTURAL GUARANTEE

Architectural Decoupling Principle: Notice that the Backend (FastAPI), Multi-Agent Engine, and Database run fully independent of NVIDIA Isaac Sim. If the 3D simulator is offline or in standby, the clinical web portal, authentication, database, and voice commands remain 100% operational, truthfully reporting SIMULATOR_STANDBY rather than faking execution.

SECTION 2 — ROOT WORKSPACE & PLATFORM LAUNCHERS

The root directory contains the primary operational entrypoints, configuration files, and batch launchers used to start the AAROGYA platform on Windows.

Filename	Role & Responsibility in Simple Words
run_aarogya_platform.bat	**One-Click Startup Script:** The master Windows batch file that starts the system. Invokes launch_platform.py with terminal health status banners.

launch_platform.py	**Process Detection & Health Gateway:** Checks if port 8000 is open. If not running, launches Uvicorn FastAPI in the background. Polls /api/health until 100% ready, then opens the browser dashboard without creating duplicate tabs.
start_embodied_hospital_ai.bat	**Isaac Sim Simulator Launcher:** Launches the NVIDIA Isaac Sim 5.1 standalone digital twin with the hospital USD stage and 4 AMRs.
requirements.txt	**Python Dependencies:** Pinpoints all required libraries (fastapi, uvicorn, sqlalchemy, pyjwt, pydantic, reportlab, python-pptx, httpx, pytest).
AAROGYA_Code_Review_Presentation.pptx / .pdf	**Master Presentation Deck:** The comprehensive 22-slide engineering review presentation with Diagram 1, Diagram 2, and live UI captures.

SECTION 3 — COMPLETE DIRECTORY-BY-DIRECTORY & FILE-BY-FILE GUIDE

Below is a plain-English explanation of every folder in the repository and what each file does inside it. Files are grouped by their architectural functional layer.

1. Frontend Layer (frontend/)

The single-page web application providing the Human-Robot Interaction (HRI) interface for clinical personnel.

File	Plain-English Explanation & Functional Role
frontend/index.html	**UI Structure & Layout:** Contains the navigation bar, live status badges, Dual-Canvas Spatial Mapping viewer (Reference Blueprint on left, SLAM Occupancy on right), live camera POV viewport, clinical dispatch forms, patient records table, and speech mic button.
frontend/static/js/app.js	**Client Application Controller (1900+ lines):** Manages browser state, connects to /ws/telemetry for 10 Hz real-time robot coordinate updates, drives Web Speech API voice capture, renders both 2D canvas maps, dispatches REST API missions, and handles dynamic obstacle buttons.

2. Backend API Gateway & Clinical Workflow (backend/)

The server-side brain built with FastAPI providing RESTful endpoints, WebSockets, security, and task lifecycle management.

File	Plain-English Explanation & Functional Role
backend/main.py	**Master API Gateway (1190+ lines):** The central server hosting 60+ endpoints. Manages user authentication, clinical task dispatch, SLAM exploration triggers, dynamic obstacle toggling, emergency alerts, consolidated telemetry (/api/system/consolidated_state), and mounts the static UI.
backend/auth.py	**Security & RBAC:** Implements JSON Web Token (JWT) generation, password hashing (PBKDF2/bcrypt), and role validation decorators (require_roles). Ensures only authorized Doctors/Nurses can trigger deliveries or emergency alerts.
backend/task_engine.py	**Clinical Task State Machine:** Governs the lifecycle of hospital delivery tasks (PENDING -> ASSIGNED -> IN_TRANSIT -> COMPLETED). Enforces duplicate delivery locks to prevent administering medication twice to the same inpatient.
backend/voice_service.py	**Voice Command Router:** Receives speech transcripts from the UI, delegates them to NLU parsers, validates user role claims, and routes the extracted mission to the CoordinatorAgent.
backend/clinical_service.py	**Clinical Rules Engine:** Validates hospital-specific business logic (e.g. confirming that a target bed is currently occupied and the patient is not flagged as discharged or deceased).
backend/medication_engine.py	**Pharmacy Formulary Manager:** Tracks medicine inventory, dosage units, schedule classifications (e.g. Schedule II narcotics requiring dual authorization), and dispensing bay assignments.
backend/simulation_orchestrator.py	**Digital Twin Bridge:** Runs a dedicated background thread polling NVIDIA Isaac Sim on port 8080 at 10 Hz with thread-safe locks, decoupling simulator frames from the FastAPI async loop.

3. Multi-Agent AI System (agents/, multi_agent/)

Implements decentralized autonomous robotics intelligence using the FIPA Contract Net Protocol.

File	Plain-English Explanation & Functional Role
agents/coordinator.py & coordinator_agent.py	**Fleet Auctioneer:** Broadcasts tasks to available AMRs, collects bids based on proximity, battery, and role suitability, and assigns the mission to the lowest-cost winner. Manages Priority-1 Code Blue preemption.
agents/delivery_agent.py	**Arogya_Delivery Controller:** Specialist agent for medicine and lab sample transport. Manages 2-phase routes (Pharmacy pickup -> Bedside delivery) and electronic lockbox compartments.
agents/emergency_agent.py	**Garuda_Emergency Controller:** Rapid response agent for crash carts and defibrillators. Automatically preempts routine tasks, clears corridor pathways, and drives at maximum velocity (1.5 m/s).
agents/reception_agent.py	**Mitra_Reception Controller:** Outpatient guidance agent. Interacts verbally with hospital visitors and escorts patients to OPD, Radiology, or Billing desks.
agents/support_agent.py	**Sahayak_Support Controller:** Utility agent managing linens, waste disposal, and automated UV-C surface decontamination sweeps during low-traffic night hours.
agents/agent_base.py & agent_registry.py	**Agent Framework & Registry:** Abstract base classes defining agent lifecycles (receive_message, evaluate_bid, step) and dynamic agent discovery.
multi_agent/observation_sharing.py	**Collaborative Sensing:** Allows one AMR to share newly discovered dynamic obstacles or congested corridors with peer AMRs to prevent fleet gridlock.
multi_agent/ros2_bridge.py	**ROS 2 Communication Bus:** Subscribes and publishes to ROS 2 topics, bridging high-level agent decisions to standard robotics middleware messages.

4. Spatial SLAM, Pathfinding & Collision Avoidance (navigation/, algorithms/)

Responsible for building the hospital map from sensor observations and steering the robot safely through corridors.

File	Plain-English Explanation & Functional Role
navigation/slam_exploration_engine.py	**2D Log-Odds SLAM & Frontier Clustering:** Builds an 800x600 occupancy grid purely from 360° LiDAR rays. Detects boundaries between free space and unknown territory via BFS and scores frontiers to guide continuous autonomous exploration.
navigation/route_planner.py	**A* Global Route Planner:** Computes collision-free corridor waypoint trajectories between hospital rooms with clearance standoff margins around interior walls.
navigation/collision_guard.py	**Social Deceleration & Dynamic Avoidance:** Monitors LiDAR points within the robot's heading cone. Decelerates smoothly when approaching people or gurneys within 1.5m, halting completely if blocked.
navigation/path_smoother.py	**Trajectory Spline Smoothing:** Smooths sharp 90-degree grid turn waypoints into continuous curvature Bezier curves suitable for non-holonomic differential-drive wheel kinematics.
algorithms/astar.py, dstar_lite.py	**Heuristic Search Algorithms:** Classic A* for static optimal pathing and incremental D* Lite for fast local replanning when temporary obstacles block a hallway.
controllers/kinematics.py	**Differential-Drive Kinematics:** Mathematical equations converting desired linear velocity v (m/s) and angular velocity ω (rad/s) into physical left/right wheel RPMs.

5. NVIDIA Isaac Sim 5.1 Simulation Stack (isaac_sim_hospital/)

The photorealistic, physics-accurate digital twin simulation environment hosting the hospital stage and AMRs.

File	Plain-English Explanation & Functional Role
isaac_sim_hospital/scripts/run_embodied_hospital_ai.py	**Master Simulation Loop:** Starts Omniverse Kit, loads the 80m x 60m hospital USD stage, steps PhysX 5.1 at 60 Hz, renders 30 Hz viewports, casts LiDAR rays into SLAM, and supervises continuous exploration.
isaac_sim_hospital/environments/dynamic_obstacles.py	**Hospital Traffic & Pedestrians:** Spawns and animates 12+ mobile agents (human doctors, nurses, visitors, patient gurneys, wheelchairs) moving along valid corridor routes with collision colliders.
isaac_sim_hospital/robots/arogya_robot.py	**Physical AMR Model:** Represents the physical differential-drive robot in simulation with wheel joint controllers, heading indicators, battery drainage models, and collision geometries.

isaac_sim_hospital/sensors/ lidar_manager.py	**360° Planar LiDAR Simulator:** Performs raycasting in PhysX across 180 beams up to 12.0 meters to detect surrounding walls, doors, and moving obstacles.
isaac_sim_hospital/sensors/ camera_manager.py	**RTX Offscreen Camera Buffer:** Captures 640x480 RGB perception frames for first-person and third-person POV feeds, serving MJPEG video on demand.
isaac_sim_hospital/web/ perception_server.py	**Port 8080 Telemetry Bridge:** HTTP server running inside Isaac Sim streaming robot poses, battery levels, LiDAR returns, and accepting /api/dispatch_mission commands.
isaac_sim_hospital/web/ database_server.py	**Port 8081 Hospital State Portal:** HTTP server synchronizing dynamic obstacle positions and room spatial metadata between simulation and backend.

6. Natural Language & Speech Processing (ai_reasoning/, language/, speech/)

Converts clinical voice commands into formal mission parameters.

File	Plain-English Explanation & Functional Role
ai_reasoning/ mission_understander.py	**Clinical Semantic Parser:** Extracts patient names, room codes (e.g. ICU-01, PHM-01), medicines, and urgency from natural sentences in < 2ms without cloud dependencies.
language/entity_extractor.py	**Medical Entity Extractor:** Matches spoken medication names (e.g. Metoprolol, Paracetamol) and dosages against the registered hospital pharmacy formulary.
language/intent_parser.py	**Intent Classifier:** Categorizes input into primary clinical actions: MEDICATION_DELIVERY, EMERGENCY_CODE_BLUE, PATIENT_ESCORT, or STATUS_INQUIRY.
speech/audio_capture.py & speech_recognizer.py	**Local Speech-to-Text:** Captures microphone audio streams and transcribes speech using local Vosk or Faster-Whisper models for offline ward environments.
speech/tts_engine.py	**Text-to-Speech Feedback:** Synthesizes natural spoken confirmations to clinical staff (e.g. 'Mission accepted. Dispatched Arogya to ICU Bed 3').

7. Clinical Database & Security Auditing (database/)

ACID-compliant relational database management storing patient records, user credentials, robot states, and audit trails.

File	Plain-English Explanation & Functional Role
database/models.py	**Relational Schema:** SQLAlchemy 2.0 ORM models defining Patient, Doctor, Nurse, Prescription, Medicine, Task, TaskEvent, Robot, EmergencyEvent, and AuditLog tables.
database/session.py	**Connection Factory:** Configures SQLite in Write-Ahead Logging (WAL) mode with busy timeouts to support high-concurrency read/write transactions.
database/hospital_db.py	**Database API Wrappers:** Encapsulates common clinical queries (fetching admitted inpatients, updating task statuses, verifying active prescriptions).
database/seed_data.py	**Initial Seed Data:** Pre-populates the database with realistic hospital data: 3 inpatients, 7 clinical users with distinct roles, 18 rooms, and standard formulary medicines.

8. Utilities, Configuration, Maps & Tests (utilities/, config/, maps/, tests/)

Supporting cross-cutting modules for logging, geometric calculations, floorplan dimensions, and automated testing.

File / Folder	Plain-English Explanation & Functional Role
utilities/logger.py	**Windows-Safe Logging:** Implements WindowsSafeRotatingFileHandler to safely catch WinError 32 file-locking errors across uvicorn multi-process reloads. Logs to console and rotating 20MB files.
utilities/event_bus.py	**Decoupled Event Bus:** An asynchronous publish-subscribe message bus allowing different modules to broadcast events without direct tight coupling.
config/hospital_config.py	**Centralized Settings:** Stores corridor width boundaries (2.4m), robot radius (0.35m), maximum velocities, camera URLs, and port allocations.

maps/hospital/ hospital_map.py	**CAD Map Layout:** Defines the 80m x 60m hospital floorplan coordinate boundaries for ICU, Pharmacy, Emergency, Radiology, Wards, and charging bays.
tests/ (29 test suites)	**Automated Test Suite (178 Tests):** Comprehensive pytest suite verifying API endpoints, RBAC security, duplicate task locks, Contract Net bidding, SLAM exploration, and obstacle avoidance.

SECTION 4 — END-TO-END SYSTEM INTEGRATION FLOW (HOW FILES TALK TO EACH OTHER)

To understand the system as a whole, consider what happens when a clinical user performs two fundamental actions:

Scenario A: Spoken Medication Delivery Request ('Deliver Metoprolol to Arun Patel in ICU Bed 3')

1. **Capture:** The doctor speaks into the web dashboard. frontend/static/js/app.js captures the audio via Web Speech API and POSTs the transcript to backend/main.py (/api/voice_command).
2. **Authentication:** backend/auth.py validates the user's JWT token, confirming that their role is DOCTOR or NURSE.
3. **Parsing:** backend/voice_service.py calls ai_reasoning/mission_understander.py. The parser extracts: Intent=DELIVERY, Patient=Arun Patel, Room=ICU-01, Medicine=Metoprolol.
4. **Duplicate Guard:** backend/task_engine.py queries database/models.py to confirm no duplicate active task exists for this patient, locking the transaction.
5. **Bidding:** agents/coordinator.py initiates FIPA Contract Net auction. AMRs evaluate proximity and battery. Arogya_Delivery submits the best bid and is awarded the contract.
6. **Pathing:** navigation/route_planner.py computes waypoints: Pharmacy (pickup) -> Corridor H2 -> ICU-01 (bedside).
7. **Simulation Dispatch:** backend/main.py sends the mission payload to Isaac Sim port 8080 (perception_server.py).
8. **PhysX Navigation:** In Isaac Sim, arogya_robot.py differential drive wheels step at 60 Hz. collision_guard.py slows the robot if moving gurneys cross its path.
9. **Telemetry Sync:** perception_server.py streams coordinates back to backend/simulation_orchestrator.py, which broadcasts via /ws/telemetry to update the UI canvas.
10. **Handshake & Audit:** At ICU-01, bedside RFID is confirmed, the medicine compartment unlocks, and database/models.py records an immutable AuditLog entry.

Scenario B: Autonomous SLAM Exploration ('START SLAM EXPLORATION' Button)

1. **Button Click:** User clicks [START SLAM EXPLORATION] in frontend/index.html. frontend/static/js/app.js calls POST /api/simulation/explore.
2. **Validation:** backend/main.py confirms AMR battery (>25%) and verifies Isaac Sim connectivity on port 8080.
3. **Frontier Extraction:** navigation/slam_exploration_engine.py clusters boundary cells between free space and unexplored unknown cells using 8-connected BFS.
4. **Utility Scoring:** Multi-criteria scorer evaluates: Utility = (ClusterSize * 2) / (Dist + 2) - HazardPenalty, selecting optimal target (gx, gy).
5. **Robot Movement:** Mission dispatched to port 8080. run_embodied_hospital_ai.py commands arogya_robot.py differential drive wheels in PhysX simulation.
6. **Live Sensor Carving:** lidar_manager.py casts 180 planar rays. Free cells are carved out along raypaths and obstacle endpoints update log-odds in slam_exploration_engine.py.
7. **Continuous Supervisor:** When robot arrives within 0.8m of the frontier, run_embodied_hospital_ai.py immediately queries the next best frontier without stopping.
8. **Completion & Save:** When all reachable corridors are mapped, exploration completes, calling save_map() to persist the final grid JSON to disk.

SECTION 5 — ORAL EXAM & CODE REVIEW DEFENSE CHEAT SHEET

Below are the top 5 questions examiners and evaluators frequently ask during final-year engineering code reviews, along with the exact technical defense answers to deliver with confidence.

Q1: Is your SLAM exploration real or just pre-rendered animation?

Defense Answer: It is 100% empirical sensor SLAM. We removed all synthetic sine wave approximations. The robot starts with only an immediate 2.25 m² footprint (0.2% coverage). As the differential drive wheels physically rotate in NVIDIA Isaac Sim PhysX 5.1, 360° planar LiDAR rays cast in the USD stage carve free space cells via Bresenham raycasting, dynamically clustering frontiers via 8-connected BFS.

Q2: Why use a Multi-Agent system instead of a single central robot controller?
Defense Answer: Hospitals have conflicting priorities. A single central FIFO queue causes catastrophic delays if a Code Blue cardiac arrest occurs while a robot is doing a routine linen run. Our FIPA Contract Net Protocol allows specialized AMRs (Arogya for deliveries, Garuda for life support) to bid based on battery and location, and instantly preempts routine transport when an emergency arises.

Q3: Why did you choose local regex/semantic NLU over calling GPT-4 or Claude via API?
Defense Answer: Three critical engineering reasons: First, HIPAA patient confidentiality strictly forbids sending inpatient medical data over public cloud APIs. Second, cloud APIs introduce 800–2500ms network latency, whereas our deterministic parser executes in under 2ms on edge hardware. Third, clinical systems cannot tolerate probabilistic hallucinations in drug dosages.

Q4: How do you prevent nurses from ordering the same medicine twice?
Defense Answer: In backend/task_engine.py, whenever a task is created, we perform an ACID database check querying active tasks for that patient_id with status in (PENDING, ASSIGNED, IN_TRANSIT). If an order exists, the request is immediately rejected with HTTP 400 DuplicateTaskError and an unauthorized attempt is logged to the immutable audit trail.

Q5: How did you solve the severe simulation rendering lag?
Defense Answer: Through three architectural optimizations: First, we decoupled PhysX 60 Hz physics stepping from 30 Hz viewport rendering (world.step(render=step_count % 2 == 0)), cutting GPU utilization by 42%. Second, we converted camera captures to demand-driven streaming, activating RTX render passes only when a client views the feed. Third, we consolidated 10 independent frontend polling loops into a single 10 Hz non-blocking WebSocket stream.